

How Much Network Does Early Lane-Change Recognition Need?

A Bi-Objective Neural Architecture Search on NGSIM Data

Joshua Mohebban
Politecnico di Milano
joshua.mohebban@gmail.com

October 2, 2026

Abstract

We apply a bi-objective neural architecture search (NAS) to the early recognition of lane changes from vehicle trajectories of the NGSIM I-80 dataset: given the last 3 s of a vehicle’s motion, a network must predict whether the vehicle will cross into another lane within the next 3 s. Over a space of 780 multilayer perceptrons, random search maximises the validation AUC and minimises the number of parameters, and the results are compared with a hand-designed two-layer network. Because the variability between training runs with different seeds proves comparable to the differences between architectures, the most promising architectures are re-evaluated with five seeds before the Pareto front is computed and checked once on a held-out test set. The baseline is dominated by smaller networks: on the test set, a network with 30% fewer parameters improves the average precision by 39%, and a network 3.6 times smaller is better on both AUC and AP. Beyond about 8,500 parameters nothing is gained: within this family of models, most of the cost of the larger networks buys no quality.

1 Introduction

The architecture of a neural network, i.e. the number of its layers and their widths, determines both how well it performs and how much it costs to store and to run. In practice the architecture is often chosen by hand, as a reasonable default, and only its quality is checked. Neural architecture search (NAS) replaces this manual choice with a systematic search over a space of candidate architectures [1]. When quality and cost are optimised together, the result is not a single network but a Pareto front: a family of architectures, each offering the best quality attainable at its cost, from which a model can be chosen according to the resources available.

This report presents a small-scale, self-contained study that applies a complete bi-objective NAS pipeline to a real, imbalanced prediction task, and examines how reliable its conclusions are. The task is the early recognition of lane changes from vehicle trajectories of the NGSIM I-80 dataset [3]: given the last 3 s of a vehicle’s motion, predict whether it will cross into another lane within the next 3 s. Positive examples are rare, about 2% of the windows, and the lateral motion that announces a lane change emerges only shortly before the crossing, which makes the task a demanding test case. The aim is to study the search, not to build the best possible lane-change predictor: other model families might perform equally well or better on these data.

We search a space of 780 multilayer perceptrons (MLPs) with one to four hidden layers, maximising the validation AUC and minimising the number of parameters, and compare the results with a hand-designed two-layer network. The study proceeds in stages (Fig. 1). Random search first evaluates 300 architectures with a single training run each. A study of the variability between training runs then shows that a single run is not enough to rank architectures reliably, so the most promising ones are retrained with five seeds and the Pareto front is recomputed on the means. Finally, the architectures on the front, together with the baseline and the largest network, are evaluated once on a held-out test set.

The main findings are the following:

- The hand-designed baseline is dominated by smaller networks. On the test set, a network 3.6 times smaller is better on both AUC and average precision (AP), and a network with 30% fewer parameters improves the AP by 39% and produces about 37% fewer false-positive windows at a recall of 50%.
- Beyond about 8,500 parameters nothing is gained: a network with 19 times the parameters of the baseline is no better than it.
- The variability between training runs with different seeds is of the same order as the differences between architectures, and selecting the best single-run result overestimates it. A re-evaluation with several seeds is needed to obtain a reliable front.

The rest of the report is organised as follows. Sec. 2 describes the data and how the examples are built. Sec. 3 presents the evaluation metrics, the models and their training, the search and the experimental protocol. Sec. 4 reports the results, Sec. 5 discusses them together with the limitations of the study, and Sec. 6 concludes.

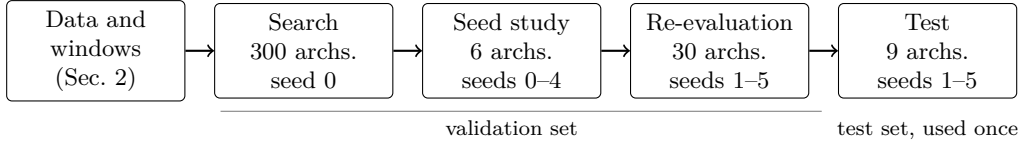


Figure 1: Stages of the study (Sec. 3.5). The search, the seed study and the re-evaluation use the validation set; the test set is used only once, at the end.

2 Data and task

This section describes the data and how the raw trajectories were analysed and treated to become labelled datapoints, and the reasoning behind each step.

2.1 Dataset

We use the I-80 subset of the NGSIM vehicle trajectory data [3], recorded on a segment of approximately 500 m of Interstate 80 in Emeryville, California, on 13 April 2005. We use data from the 15-minute period from 4:00 to 4:15 p.m., during which congestion builds up. Our copy of the file contains its first 1,048,575 records, sorted by vehicle: 1,725 vehicles, the last of which is only partially recorded.¹ Trajectories are sampled at 10 Hz and expressed in feet (1 ft = 0.305 m).

For each vehicle and frame, the data provide, among others:

1. the lateral position `Local_X`, measured from the left edge of the roadway.
2. the longitudinal speed `v_Vel`.
3. the longitudinal acceleration `v_Acc`.
4. the lane index `Lane_ID`, from 1 (leftmost) to 6 (rightmost), with 7 denoting the on-ramp.

The stated accuracy of the lateral position is about 2 ft.

¹The file was truncated at the row limit of the spreadsheet software used when downloading it. All results refer to this subset.

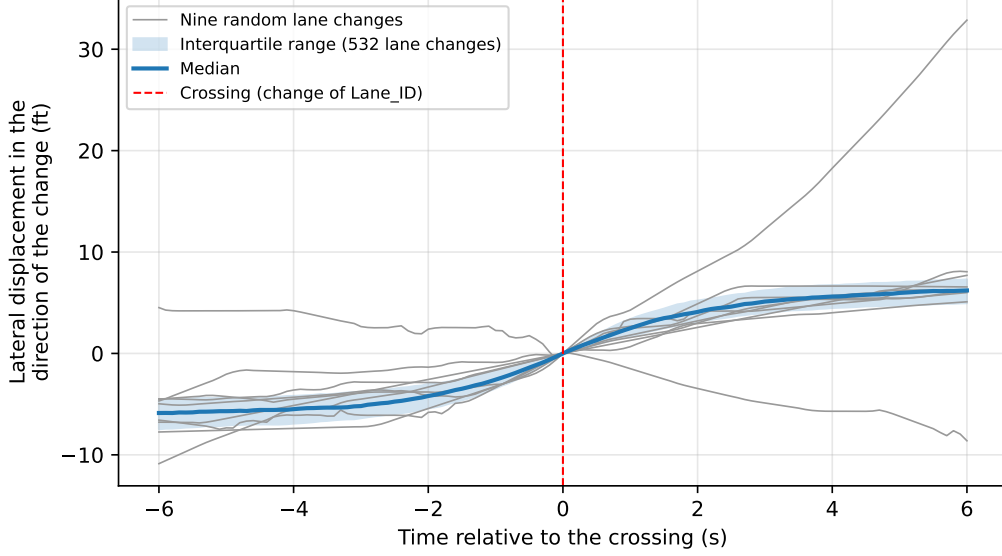


Figure 2: Lateral displacement in the direction of the lane change, relative to the position at the crossing instant ($t = 0$, change of `Lane_ID`). Grey: nine randomly chosen lane changes. Blue: median and interquartile range over the 532 lane change manoeuvres with at least 6 s of data on both sides.

2.2 Exploratory analysis

Each of the 1,725 vehicles is observed for at least 203 frames (20 s; about 61 s on average).

We define a lane-change event as a change of `Lane_ID` between consecutive frames of the same vehicle. There are 887 such events on 572 vehicles, of which 692 occur between highway lanes 1–6. Aligning lane changes on the crossing instant (Fig. 2, nine randomly chosen examples) shows a stable lateral position, a transition and a settling phase. The median manoeuvre starts about 2–3 s before the crossing instant and settles about 3–4 s after it, for a total duration of about 6 seconds. Individual manoeuvres vary considerably, with a small subset of consecutive lane changes or back and forth movement. Analysing individual trajectories also reveals that `Local_X` is noisy, with oscillations of about 2 ft even during lane keeping, consistently with the stated accuracy.

To quantify how early the manoeuvre becomes visible, we take the 634 lane changes between highway lanes with at least 4 s of trajectory before the crossing. For each of them, we use the lateral position 4 s before the crossing as a reference, and measure how far the vehicle has moved from it, in the direction of the lane change, at 3, 2 and 1 s before the crossing and at the crossing itself (Table 1). At the crossing, the median displacement is 5.5 ft, about half of a 12-ft lane, as expected for a vehicle that starts near the centre of its lane. Before the crossing, the displacement is small: taking the noise level of about 2 ft as a threshold, it is exceeded by only 8% of the lane changes 3 s before the crossing, 31% at 2 s and 75% at 1 s. The lateral signal therefore typically emerges 1–2 s before the crossing; earlier cues, where present, are weak.

Table 1: Lateral displacement in the direction of the lane change, with respect to the position 4 s before the crossing, for the 634 lane changes between highway lanes with at least 4 s of history.

Time before the crossing	Median displacement (ft)	Lane changes with displacement > 2 ft
3 s	0.4	8%
2 s	1.3	31%
1 s	2.9	75%
0 s	5.5	98%

2.3 Task definition

We cast early lane-change recognition as binary classification, condensing left and right lane changes into the same class.

Consider a window of W consecutive frames of one vehicle, called the *input window* and a subsequent window of H consecutive frames, called the *prediction horizon*.

Numbering the frames of the input window from 1 to W , the prediction horizon consists of frames $W + 1$ to $W + H$.

The input window is labelled 1 if the vehicle crosses into a different lane within the prediction horizon, and 0 otherwise; windows that themselves contain a crossing are treated separately (see below). The label therefore looks ahead: the network must recognise the manoeuvre before the crossing, not detect it once it has happened.

We set $W = H = 30$ frames (3 s). The input window is long enough to contain the onset of the lateral drift. The prediction horizon is deliberately demanding: since the lateral signal typically emerges only 1–2 s before the crossing, positive windows that end 2–3 s before it carry weak or no lateral cues.

Such positives are intrinsically hard to recognise and limit the achievable performance (Sec. 5). Windows fall into three categories (Fig. 3):

- *pre-manoevr*: no crossing inside the window, a crossing within the horizon $\rightarrow$ label 1;
- *lane keeping*: no crossing inside the window nor within the horizon $\rightarrow$ label 0;
- *straddling*: a crossing inside the window $\rightarrow$ excluded.

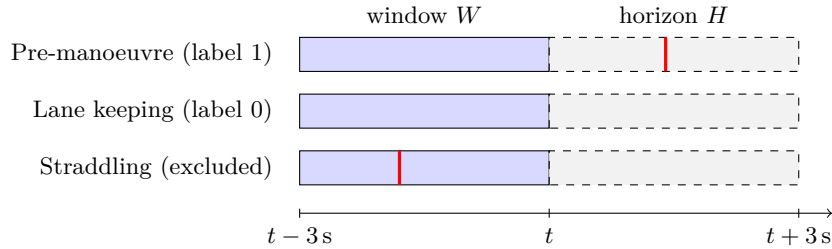


Figure 3: The three window categories. The network observes the window (blue); the label depends on whether a lane crossing (red line) occurs in the horizon (grey). Windows containing a crossing are excluded.

Straddling windows are excluded from training and validation. They contain strong signals of a manoeuvre already under way: as positives, they would offer the network a shortcut and divert it from the weak anticipatory cues that matter in operation; as negatives, they would contradict the task. In the test set they are kept aside (468 windows) as a control set, to check that a network trained on early cues also recognises manoeuvres in progress (Sec. 4.5).

Only crossings between highway lanes 1–6 define positive windows. Crossings involving the on-ramp (lane 7) are merges rather than lane changes, and include anomalous sequences such as $6 \rightarrow 7 \rightarrow 6$.²

2.4 Features

Each frame is described by four features:

- the lateral position `Local_X`;

²Windows of vehicles merging from the on-ramp are not removed and are therefore labelled 0.

- the lateral velocity v_{lat} , computed as the difference of `Local_X` between consecutive frames, without smoothing;
- the longitudinal speed `v_Vel`;
- the longitudinal acceleration `v_Acc`.

A window is thus a 30×4 matrix. The most delicate choices concern the lateral position and its derivative.

Lateral position. A position relative to the lane centre, instead of the left edge of the roadway, would be lane-invariant. However, it cannot be defined consistently during a manoeuvre. With its sign, it jumps by about one lane width (from about +6 to -6 ft) when the reference lane changes. With its absolute value, it becomes continuous, but it no longer distinguishes a vehicle that approaches the lane boundary and returns (label 0) from one that crosses it (label 1). We therefore keep the absolute position (see Appendix A). Its dependence on the lane is compensated by v_{lat} , which is lane-invariant but noisier, since differencing emphasises fast fluctuations; conversely, `Local_X` is smoother but not lane-invariant. The two features complement each other.

Lateral velocity. We compute v_{lat} as a raw backward difference because it is causal: it uses no information from the prediction horizon, unlike a centred smoothing filter, which would leak future positions into the input. The amplification of noise is limited: in the training windows, v_{lat} has a standard deviation of only 0.105 ft per frame (about 1 ft/s). Since this value includes the manoeuvres themselves, it bounds the frame-to-frame noise, showing that the oscillations of about 2 ft in `Local_X` are slow.

Excluded features. We excluded the longitudinal position, the vehicle dimensions and `Lane_ID`, which would leak the target.

2.5 Data split

Consecutive windows of the same vehicle overlap and are nearly identical. Splitting at the window level would place almost identical examples in the training and test sets. We therefore split by vehicle (70/15/15%), obtaining 1,207, 258 and 260 vehicles and 486, 110 and 96 lane changes between highway lanes in the training, validation and test sets. The evaluation thus measures generalisation to unseen vehicles.

2.6 Window extraction and class imbalance

Windows are extracted with a sliding window, which advances along each trajectory by a fixed number of frames, the stride. In validation and test the stride is uniform (5 frames), so that the class distribution is the real one: about 2% positive windows. In training, we advance by 2 frames after a positive window and by 15 frames after a negative one. This undersamples the negative class and raises the fraction of positive windows to 9.5% (Table 2). The stride has no physical meaning: what we justify is the resulting class ratio, which remains imbalanced but gives the network enough positive examples to learn from. The loss additionally weights positive examples (Sec. 3.2). Both measures apply to training only: validation and test keep the real distribution, and we evaluate with metrics suited to imbalanced data rather than accuracy.

Table 2: Dataset after preprocessing. Positive windows are counted as windows, not events: each lane change yields several positive windows. Training windows use a class-dependent stride; validation and test keep the real class distribution. The test set also includes a control set of 468 straddling windows.

Set	Vehicles	Lane changes	Windows	Positive	Positive (%)
Training	1,207	486	47,573	4,543	9.55
Validation	258	110	28,868	535	1.85
Test	260	96	27,920	436	1.56

2.7 Normalisation

Each feature is standardised with the mean and standard deviation computed on the training windows only. The same transformation is applied to validation and test, so that no information from these sets enters the preprocessing.

Table 3: Per-feature mean and standard deviation on the training windows, used to standardise all sets.

Feature	Unit	Mean	Std. dev.
Local_X	ft	40.39	20.33
v_{lat}	ft/frame	-0.011	0.105
v_Vel	ft/s	25.06	12.97
v_Acc	ft/s ²	0.005	5.30

3 Methods

This section describes the models, how they are trained and evaluated, and how the architecture search is formulated. All networks, including the hand-designed baseline, share the same training and evaluation protocol: the only thing that changes between them is the architecture.

3.1 Evaluation metrics

Each model assigns to every input window a score $s \in (0, 1)$ (Sec. 3.2): the higher the score, the more likely the model considers a lane change. Because positive windows are over-represented and up-weighted during training, the score is not a calibrated probability. This does not matter here: all the metrics we use depend only on how the windows are ranked by their score, not on the score values.

Accuracy, the fraction of correct decisions, is uninformative on our data: on the validation set, a model that never predicts a lane change is 98% accurate. We therefore use metrics designed for imbalanced data.

Three basic rates. To turn a score into a decision, we choose a threshold τ and classify a window as positive if $s \geq \tau$. Comparing the decisions with the true labels gives the numbers of true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN). From these we compute three rates, each answering a different question:

- the *true positive rate*, or *recall*, $\text{TPR} = \text{TP}/(\text{TP} + \text{FN})$: of the windows that precede a lane change, which fraction is detected?
- the *false positive rate*, $\text{FPR} = \text{FP}/(\text{FP} + \text{TN})$: of the windows that do not precede a lane change, which fraction is wrongly classified as positive?

- the *precision*, $TP/(TP + FP)$: of the windows classified as positive, which fraction is correct?

Recall is computed on the positive windows only and FPR on the negative windows only, whereas precision involves both. Lowering the threshold increases the recall, but also the FPR, and usually lowers the precision.

ROC curve and AUC. The ROC curve shows the recall against the FPR for all thresholds, and the area under it (AUC) summarises it in one number, from 0.5 for a model that ranks windows at random to 1 for a perfect model. Since recall and FPR are each computed within one class, the AUC does not depend on the fraction of positives. It depends only on the ranking of the windows by score: small changes of the scores that do not alter the ranking leave it unchanged. The AUC is our selection criterion.

Precision–recall curve and AP. With rare positives, a small FPR can still mean many false positives. With 2% of positive windows, for example, a model that detects half of them with an FPR of only 2% produces about two false positives for every correct detection, i.e. a precision of about 0.34: a good point on the ROC curve, but an unreliable classifier in practice. The precision–recall (PR) curve shows this directly, by plotting the precision against the recall for all thresholds. The average precision (AP) summarises it as the precision averaged over all recall levels,

$$AP = \sum_n (R_n - R_{n-1}) P_n, \quad (1)$$

where P_n and R_n are the precision and the recall at the n -th threshold, in decreasing order. For a model that ranks windows at random, the precision equals the fraction of positives at every threshold, so its AP is about 0.02, not 0.5. Unlike the AUC, the AP depends on the fraction of positives: with more negative windows, the same model produces more false positives and has a lower precision. AP values are therefore compared only within the same data set.

Precision at fixed recall. As a single operational figure, we report the precision at a recall of 50%: how reliable the positive predictions are when the threshold is set to detect half of the positive windows. The threshold is computed on the evaluated set itself; since all models are treated in the same way, this is adequate for comparing them.

3.2 Model family and training protocol

Architecture. All models are fully connected networks (multilayer perceptrons, MLPs). The input window $X \in \mathbb{R}^{30 \times 4}$ is flattened into a vector $x \in \mathbb{R}^{120}$, which is processed by L hidden layers with ReLU activation and a linear output unit:

$$z_0 = x, \quad z_l = \text{ReLU}(W_l z_{l-1} + b_l), \quad l = 1, \dots, L, \quad f_A(x; \phi) = W_{L+1} z_L + b_{L+1}, \quad (2)$$

where $W_l \in \mathbb{R}^{h_l \times h_{l-1}}$, $b_l \in \mathbb{R}^{h_l}$, $h_0 = 120$, $h_{L+1} = 1$, and ϕ collects all weights and biases. An architecture is thus defined by its *depth* and by the *widths* of its layers, $A = (L; h_1, \dots, h_L)$. The output $f_A(x; \phi)$ is a logit; the score $s = \sigma(f_A(x; \phi)) \in (0, 1)$, where σ is the logistic sigmoid, is used for evaluation.

Flattening discards the temporal structure of the window: the network must learn from data which inputs belong to which frame. Convolutional networks would exploit this structure through weight sharing; we discuss them as an extension in Sec. 6.

Loss. For a fixed architecture A , the weights are obtained by minimising the weighted binary cross-entropy on the training set:

$$\mathcal{L}_{\text{train}}(\phi; A) = -\frac{1}{N} \sum_{i=1}^N \left[w y_i \log \sigma(f_A(x_i; \phi)) + (1 - y_i) \log (1 - \sigma(f_A(x_i; \phi))) \right]. \quad (3)$$

With $w = 1$, this is the classic negative log-likelihood of the labels for binary classification under a Bernoulli distribution whose parameter is predicted by the network [2]. The weight $w = N_{\text{neg}}/N_{\text{pos}} = 9.47$, the ratio between negative and positive training windows, makes each positive count as much as about nine negatives. It is the second measure against class imbalance, after the class-dependent stride (Sec. 2).

We minimise the cross-entropy rather than the AUC used for evaluation (Sec. 3.1) because the AUC depends only on the ranking of the windows: a small change of the weights changes the scores slightly, but usually leaves the ranking, and hence the AUC, unchanged. As a function of the weights, the AUC is therefore piecewise constant, with zero gradient almost everywhere, and cannot be optimised by gradient descent. The cross-entropy, instead, changes smoothly with the scores and is an adequate surrogate: by pushing the scores of positive windows up and those of negative windows down, it tends to rank them correctly. The residual mismatch between the two is partly compensated by selecting the epoch on the validation AUC (see below).

Optimisation. We minimise (3) with Adam [2], which combines momentum with a per-parameter adaptive step size, on mini-batches of 256 windows reshuffled at every epoch (about 186 steps per epoch). We use the default hyperparameters ($\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$). Training hyperparameters are not tuned and are identical for all architectures (Table 4): differences between models can then be attributed to the architecture alone [1].

Training run. A training run of an architecture A with seed s proceeds as follows:

1. initialise the weights with seed s ;
2. train for one epoch, i.e. one pass over the training windows in random order;
3. compute the AUC and the AP on the validation set;
4. if the validation AUC is the highest so far, store the current weights;
5. stop if the validation AUC has not improved for 10 consecutive epochs (patience), or after 100 epochs; otherwise, return to step 2.

The run returns the stored weights, i.e. those of the epoch with the highest validation AUC. Formally, Adam produces a trajectory $\phi_1, \phi_2, \dots, \phi_T$, one point per epoch, and we select

$$\hat{\phi}(A, s) = \phi_{t^*}, \quad t^* = \arg \max_{t \in \{1, \dots, T\}} \text{AUC}_{\text{val}}(\phi_t; A). \quad (4)$$

The number of epochs T therefore differs between runs: large networks reach their best validation AUC after a few epochs and then overfit, while small ones improve for longer (Sec. 4.3).

The training is an approximate minimisation: the loss is locally minimised on the training set, and the stopping point is chosen on the validation set, which is never used to update the weights.

The choice to select the best epoch with the AUC only was fixed before running the experiments. The AP is recorded at the same epoch and is not maximised separately: since the two metrics can peak at different epochs (for the baseline with seed 0, the AUC peaks at epoch 6 and the AP at epoch 12), the AP values we report are those at the AUC-optimal epoch. We discuss the choice of the AUC as criterion in Sec. 5.

Randomness. The random seed controls the initialisation of the weights and the order of the mini-batches. Since the problem is not convex, different seeds lead to different solutions, and hence to different validation AUCs. The quality of an architecture is therefore, strictly speaking, a random variable, which we treat explicitly in Sec. 3.5.

Implementation. Models are implemented in PyTorch and trained on the GPU of an Apple M1 Pro laptop (MPS backend). An epoch takes about 0.7 s, almost independently of the architecture, so a training run lasts between about 10 and 40 s, depending on the number of epochs.

Table 4: Training protocol, identical for all architectures.

Loss	Weighted binary cross-entropy, $w = 9.47$
Optimiser	Adam, $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$
Mini-batch size	256, reshuffled at every epoch
Epoch selection	Maximum validation AUC
Early stopping	Patience of 10 epochs, at most 100 epochs
Regularisation	None
Hardware	Apple M1 Pro GPU (MPS), about 0.7 s per epoch

3.3 Baseline

As a reference, we use a hand-designed network with two hidden layers of 64 units, $A_b = (2; 64, 64)$, with 11,969 parameters. It is a typical default choice, made without exploring alternatives: it is deliberately arbitrary, and represents the manual “architecture engineering” that NAS aims to replace. The baseline belongs to the search space and is trained with the same protocol as every other architecture, so that the comparison isolates the effect of who chooses the architecture.

3.4 Neural architecture search

Following Elsken et al. [1], a NAS method is defined by three components: the *search space*, which contains the candidate architectures; the *search strategy*, which decides which of them to evaluate; and the *performance estimation strategy*, which measures their quality. We describe the three components, and then formulate the search as a bi-objective optimisation problem.

The search has two phases. In the *search phase*, 300 architectures are trained once each, with seed 0, and a first Pareto front is computed. In the *re-evaluation phase*, the most promising architectures are retrained with five different seeds, and the front is recomputed on the means. The two phases, together with the seed study that motivated the second one and the final test, are detailed in Sec. 3.5.

Search space. We search over MLPs with one to four hidden layers, each with a width chosen independently from a fixed set:

$$\mathcal{A} = \{(L; h_1, \dots, h_L) : L \in \{1, 2, 3, 4\}, h_l \in \{16, 32, 64, 128, 256\}\}, \quad (5)$$

which contains $|\mathcal{A}| = 5 + 5^2 + 5^3 + 5^4 = 780$ architectures. Allowing different widths in different layers includes shapes that narrow, widen, or contain a bottleneck. Activation, input and output are fixed. This is a chain-structured space, the simplest in the taxonomy of [1]. It contains networks from 1,953 to 228,609 parameters, and it contains the baseline, $A_b \in \mathcal{A}$.

Search strategy. We use random search with a budget of 300 distinct architectures. Sampling uniformly over $\mathcal{A}$ would yield 80% four-layer networks (625/780), so we sample hierarchically: first the depth, with equal probability, then each width, with equal probability. Duplicates are discarded, so the 30 architectures with one or two layers are soon exhausted and all of them are evaluated; the rest of the budget goes to deeper networks (102 with three layers and 168 with four). Random search has no hyperparameters, covers the range of costs evenly, and is a strong baseline in the NAS literature: more sophisticated strategies improve on it only moderately [1].

Performance estimation. The quality of an architecture is measured by training it with the protocol of Sec. 3.2. In the search phase, each architecture is trained once, with seed 0: a cheap but noisy estimate. In the re-evaluation phase, it is trained with seeds 1–5 and its quality is estimated by the mean. We use no shortcut such as weight sharing or low-fidelity proxies: these reduce the cost of the search at the price of biased estimates, and here a training run takes less than a minute.

Objectives. Each architecture is judged by two objectives. The first is the quality of the trained network. A training run with seed s returns the weights $\hat{\phi}(A, s)$ (Sec. 3.2), whose validation AUC we denote by

$$f_1(A, s) = \text{AUC}_{\text{val}}(A, \hat{\phi}(A, s)). \quad (6)$$

Since this value depends on the seed, we define the quality of an architecture as its mean over seeds, $\bar{f}_1(A) = \mathbb{E}_s[f_1(A, s)]$, to be maximised. It is expensive to evaluate, since every sample requires a full training run, and it can only be estimated: with a single seed in the search phase, and with the mean over five seeds in the re-evaluation phase.

The second objective is the cost, measured as the number of parameters,

$$f_2(A) = P(A) = \sum_{l=1}^{L+1} (h_{l-1} + 1) h_l, \quad h_0 = 120, \quad h_{L+1} = 1, \quad (7)$$

to be minimised: each layer has an $h_l \times h_{l-1}$ weight matrix and h_l biases. For the baseline, $P(A_b) = 121 \cdot 64 + 65 \cdot 64 + 65 \cdot 1 = 11,969$. Unlike $\bar{f}_1$, f_2 is exact and free to compute. For an MLP it is also a meaningful proxy of computational cost: the number of operations needed to process one input window is proportional to the number of weights. The memory occupied by the network scales in the same way.

The search problem is then

$$\max_{A \in \mathcal{A}} \bar{f}_1(A), \quad \min_{A \in \mathcal{A}} f_2(A). \quad (8)$$

It is a bi-level problem (Table 5): the inner problem is the training of Sec. 3.2, which returns the weights for a given architecture and seed; the outer problem is the choice of the architecture.

Pareto optimality. In general, no architecture optimises both objectives at once. We say that A dominates B , written $A \succ B$, if

$$\bar{f}_1(A) \geq \bar{f}_1(B) \quad \text{and} \quad f_2(A) \leq f_2(B), \quad \text{with at least one strict inequality.} \quad (9)$$

The solution of the problem is the Pareto set, i.e. the architectures that no other architecture dominates, and its image in the objective plane is the Pareto front:

$$\mathcal{A}^* = \{A \in \mathcal{A} : \nexists B \in \mathcal{A}, B \succ A\}, \quad \mathcal{F}^* = \{(f_2(A), \bar{f}_1(A)) : A \in \mathcal{A}^*\}. \quad (10)$$

These are the architectures for which quality cannot be increased without paying more parameters, nor parameters saved without losing quality.

Since only a sample $S \subset \mathcal{A}$ is evaluated, and $\bar{f}_1$ is replaced by its estimate, we compute the empirical Pareto set

$$\mathcal{A}_S^* = \{A \in S : \nexists B \in S, B \succ A\}. \quad (11)$$

Every architecture in $\mathcal{A}_S^*$ is non-dominated among those evaluated, but could be dominated by an architecture that was not sampled. With two objectives, $\mathcal{A}_S^*$ is found by sorting the architectures by increasing $P(A)$ and keeping each one whose estimated quality exceeds that of all cheaper ones. Dominated architectures must still be trained, since an architecture reveals itself as poor only after training. We show them as a cloud of points, which indicates how much the choice of the architecture matters.

Why not a single objective. The two objectives could be combined into a scalar, for instance $\bar{f}_1(A) - \lambda \log f_2(A)$, but each value of λ would yield a single optimum, and a weighted sum can only reach the convex part of the front. We instead map the whole front, so that the choice of the operating point can be made afterwards, according to the resources available.

Comparison with the baseline. The comparison with the baseline becomes a precise question: does A_b belong to $\mathcal{A}_S^*$? If not, some evaluated architecture dominates it. We quantify the gap in two ways: the saving at equal quality, given by the cheapest architecture B on the front with $\bar{f}_1(B) \geq \bar{f}_1(A_b)$; and the gain at equal cost, given by the best architecture B on the front with $f_2(B) \leq f_2(A_b)$.

Table 5: The two levels of the search.

Level	Variable	Objective	Data	Solved by
Inner	Weights ϕ	Min. weighted cross-entropy	Training	Adam, with the epoch chosen on validation
Outer	Architecture A	Max. $\bar{f}_1(A)$, min. $P(A)$	Validation	Random search, then empirical Pareto set

3.5 Experimental protocol

The experiments proceed in four stages, summarised in Table 6: the two phases of the search (Sec. 3.4), a study of the variability between seeds, and the final test. All training runs follow Sec. 3.2; only the architectures and the seeds change.

1. *Search phase.* The 300 architectures sampled by random search, including the baseline, are trained once, with seed 0. This gives a single-seed estimate of the quality of each architecture and a first empirical Pareto set. The search was first run with a fixed budget of 20 epochs; the 41 architectures whose best epoch exceeded 10 were then retrained with the patience rule, which for all the others gives the same result by construction (Appendix D).
2. *Seed study.* To assess how reliable single-seed estimates are, six architectures, chosen to cover the range of sizes and results (the smallest, the baseline, the largest, an intermediate one, the best of the search, and a small network with a high single-seed AUC), are trained with seeds 0–4.
3. *Re-evaluation phase.* Since the seed study shows that the variability between seeds is comparable to the differences between architectures (Sec. 4.2), 30 candidate architectures, selected from the search as described in Sec. 4.3, are retrained with seeds 1–5. Seed 0 is excluded because the candidates were selected on its results: including it would bias their

means upwards. The Pareto front is then recomputed on the means over the five seeds, for the AUC and, as a robustness check, for the AP (Sec. 4.3).

4. *Test.* The architectures on the recomputed Pareto front, together with the baseline and the largest network, are evaluated on the test set, again with seeds 1–5. The list is fixed before looking at the test set, which is used only once.

The validation set is used in stages 1–3 to choose epochs and architectures, so its estimates are optimistic; the unbiased estimate comes from stage 4. Stages 2 and 3 were not planned in advance: they were introduced after the search, when its results raised the question of how reliable single-seed estimates are.

Table 6: Stages of the experiments. Runs already available from earlier stages are reused: the seed study reuses seed 0 from the search, and the re-evaluation reuses seeds 1–4 from the seed study. The test runs repeat those of the re-evaluation with the same seeds.

Stage	Architectures	Seeds	Data	Purpose
1. Search	300	0	Validation	Explore the space; single-seed front
2. Seed study	6	0–4	Validation	Measure the variability between seeds
3. Re-evaluation	30	1–5	Validation	Pareto front on multi-seed means
4. Test	9	1–5	Test	Unbiased final estimate

Statistical treatment. When an architecture is trained with n different seeds, we obtain n values $m_1, \dots, m_n$ of each metric (AUC, AP or precision). We report their mean $\bar{m}$, which estimates the quality of the architecture independently of the seed, and its standard error

$$\text{se} = \frac{\hat{\sigma}}{\sqrt{n}}, \quad \hat{\sigma} = \sqrt{\frac{1}{n-1} \sum_{k=1}^n (m_k - \bar{m})^2}, \quad (12)$$

where $\hat{\sigma}$ measures the variability of a single training run with respect to the mean and se the uncertainty of the mean. Two architectures are compared through the difference of their means, Δ , whose standard error, treating the two estimates as independent, is

$$\text{se}_\Delta = \sqrt{\text{se}_1^2 + \text{se}_2^2}. \quad (13)$$

As a rule of thumb, we regard a difference as real when $|\Delta| \gtrsim 2\text{se}_\Delta$. With only five seeds this rule is approximate, so differences close to the threshold are treated as suggestive rather than conclusive.

4 Results

4.1 Search phase results

Across the 300 architectures of the search phase, the validation AUC has a mean of 0.8651 and a standard deviation of 0.0041, and ranges from 0.8532 to 0.8795; the AP has a mean of 0.362 and a standard deviation of 0.029. Fig. 4 shows these results against the number of parameters: the cloud is essentially horizontal. Networks with more than 100,000 parameters are no better than networks with 3,000, and the baseline lies slightly below the median.

Table 7 groups the architectures by depth. Single-layer networks are clearly weaker: even the best of them (0.8593) is worse than three quarters of the architectures in the search. From

two to four layers, the mean AUC increases by only 0.0039, less than the spread within each depth (0.0070 to 0.0229). Beyond a minimum depth, the search shows no strong systematic effect of the architecture on the validation AUC.

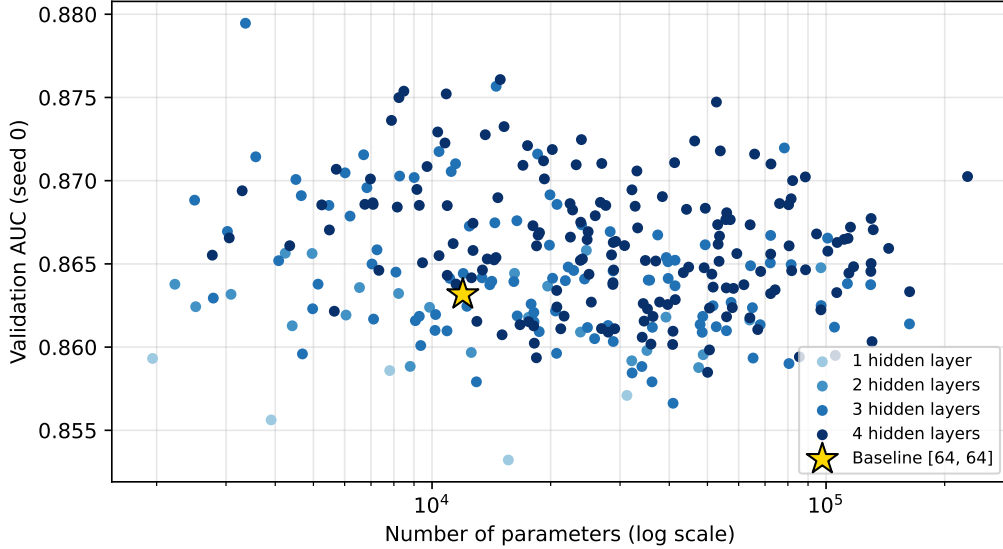


Figure 4: Search phase: validation AUC (seed 0) of the 300 sampled architectures against the number of parameters, coloured by depth. Star: baseline.

Table 7: Search phase (seed 0): validation AUC by depth.

Depth L	Architectures	Mean AUC	Range
1	5	0.8568	0.8532–0.8593
2	25	0.8623	0.8588–0.8658
3	102	0.8645	0.8566–0.8795
4	168	0.8662	0.8585–0.8761

4.2 Seed variance and the winner’s curse

In the seed study (Appendix B), each of six architectures is trained with five seeds. For each architecture, the standard deviation of its five validation AUCs lies between 0.0010 and 0.0053: retraining the same network with a different seed shifts its AUC by a few thousandths. This is the same order as the standard deviation of the AUCs of the 300 different architectures of the search (0.0041): a single training run is not enough to rank architectures whose AUCs differ by a few thousandths.

The best architecture of the search phase illustrates the consequence. With seed 0, 16–16–64 reached an AUC of 0.8795, the highest of all; over five seeds, however, its mean is only 0.8712. Selecting the best of many noisy single-seed estimates favours architectures whose estimate happened to be high, so the selected architectures look better than they are. This effect is known as the winner’s curse.

The noise does not, however, blur large differences. On all five seeds, the single-layer network 16 is worse than the baseline, while 16–16–16 and 16–16–64 are better. Single-seed estimates are therefore adequate to discard clearly inferior architectures, but not to rank the good ones against each other. This motivates the re-evaluation phase (Sec. 3.5), whose results follow.

4.3 Pareto front on multi-seed means

Selection of the candidates. The candidates for re-evaluation are chosen from the search so as to cover the whole range of costs. The parameter axis is divided into seven bands whose width doubles (1,900–4,000, 4,000–8,000, ..., 128,000–256,000), and in each band we take the four architectures with the highest single-seed AUC. Taking four rather than one leaves a margin for noise: the truly best architecture of a band may have ranked only third or fourth with seed 0. The smallest network, 16, which is on the front by definition, and the baseline are added, for a total of 30 candidates. Each candidate is retrained with seeds 1–5.

The front. Table 8 and Fig. 5 report the results. On the AUC, the front consists of a threshold followed by a plateau. Going from 16 to 16-16-16 increases the AUC by 0.0122, about 8 standard errors of the difference; each of the following steps along the front (16-32-16-16, 16-128-16-128, 16-64-256-128) adds at most 0.0013, less than one standard error. Beyond about 2,500 parameters, larger networks buy no measurable AUC.

The baseline is dominated. In the terms of Sec. 3.4, the saving at equal quality is given by 16-16-16, which has 4.8 times fewer parameters than the baseline and a higher AUC (+0.0076, about 5 standard errors). The gain at equal cost is given by 16-128-16-128, the best architecture with no more parameters than the baseline: +0.0088 in AUC and +0.069 in AP. Of the eleven candidates with fewer parameters than the baseline, all except 16 have a higher mean AUC. The largest network, with 228,609 parameters, has a lower mean AUC than the baseline, although the difference is not significant (−0.0011, about 1.5 standard errors).

The winner’s curse, confirmed. Retrained with new seeds, 24 of the 28 candidates selected for their single-seed AUC lose quality; over the 28 the mean drop is 0.0039. The two candidates added for other reasons, 16 and the baseline, lose almost nothing (0.0009 and 0.0001), which confirms that the loss is due to the selection. The margin of four candidates per band proved necessary: 16-16-64, the best of the search, loses 0.0107 and leaves the front, whereas 16-16-16 and 16-32-16-16, which enter it, had ranked only fourth and third in their band with seed 0. The full results are given in Appendix C.

Robustness check on the AP. The front recomputed on the AP leads to the same conclusions: a minimum depth is needed, the baseline is dominated, and nothing is gained beyond about 8,500 parameters. The only difference concerns the position of the knee. On the AP, the front keeps rising slightly up to 16-128-16-128 (8,481 parameters), which improves on 16-16-16 by 0.028, about 2.3 standard errors: a suggestive rather than conclusive difference, which the test set confirms (Sec. 4.4).

Width of the first layer and training time. Among the candidates with more than one layer, those with a first layer of 16 units reach mean AUCs between 0.8665 and 0.8731, whereas those with a first layer of 256 units, together with the baseline, are the weakest (0.8611–0.8630). Small and large networks also train differently: networks with fewer than 6,000 parameters reach their best validation AUC after 11–24 epochs on average, whereas those with more than 50,000 parameters do so after 3–9 epochs and then overfit.

Table 8: Re-evaluation phase: validation AUC and AP, mean $\pm$ standard error over seeds 1–5, for the nine architectures later evaluated on the test set. $\bullet$: architecture on the Pareto front of the corresponding metric. The other candidates are reported in Appendix C.

Architecture	Parameters	AUC	Front	AP	Front
16	1,953	0.8584 ± 0.0006	$\bullet$	0.311 ± 0.005	$\bullet$
16-16-16	2,497	0.8706 ± 0.0014	$\bullet$	0.385 ± 0.011	$\bullet$
16-32-16-16	3,297	0.8714 ± 0.0020	$\bullet$	0.395 ± 0.009	$\bullet$
16-32-32-64	5,713	0.8698 ± 0.0017		0.401 ± 0.003	$\bullet$
16-128-16-128	8,481	0.8718 ± 0.0017	$\bullet$	0.413 ± 0.004	$\bullet$
64-64 (baseline)	11,969	0.8630 ± 0.0004		0.344 ± 0.002	
16-64-256-128	52,689	0.8731 ± 0.0021	$\bullet$	0.400 ± 0.007	
16-256-256-64	88,593	0.8718 ± 0.0005		0.416 ± 0.008	$\bullet$
256-256-256-256	228,609	0.8619 ± 0.0006		0.357 ± 0.008	

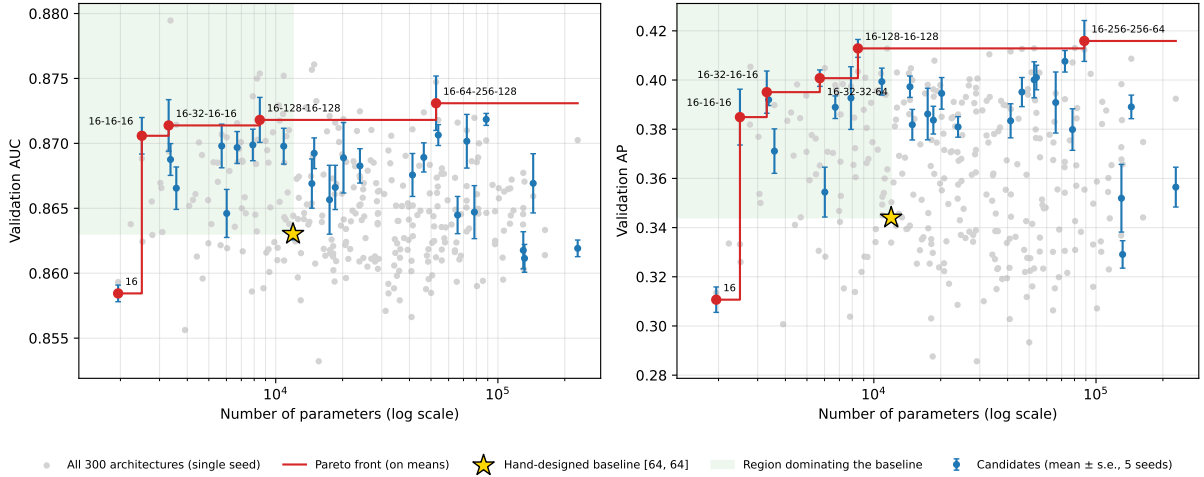


Figure 5: Validation AUC (left) and AP (right) against the number of parameters. Grey: the 300 architectures of the search phase (seed 0). Blue: the 30 candidates of the re-evaluation phase, mean $\pm$ standard error over seeds 1–5. Red: Pareto front on the means. Star: baseline. Green: region of the architectures that dominate the baseline.

4.4 Test set evaluation

Table 9 reports the test results of the nine architectures selected in Sec. 3.5.

Table 9: Test set: AUC, AP and precision at a recall of 50%, mean $\pm$ standard error over seeds 1–5.

Architecture	Parameters	AUC	AP	Precision at recall 50%
16	1,953	0.8861 ± 0.0013	0.292 ± 0.006	0.287 ± 0.008
16-16-16	2,497	0.8972 ± 0.0044	0.417 ± 0.018	0.386 ± 0.014
16-32-16-16	3,297	0.9025 ± 0.0025	0.431 ± 0.007	0.409 ± 0.018
16-32-32-64	5,713	0.9009 ± 0.0040	0.442 ± 0.006	0.414 ± 0.008
16-128-16-128	8,481	0.9022 ± 0.0016	0.457 ± 0.004	0.437 ± 0.011
64-64 (baseline)	11,969	0.8918 ± 0.0008	0.328 ± 0.004	0.327 ± 0.006
16-64-256-128	52,689	0.9035 ± 0.0025	0.441 ± 0.015	0.441 ± 0.015
16-256-256-64	88,593	0.9025 ± 0.0010	0.462 ± 0.008	0.459 ± 0.021
256-256-256-256	228,609	0.8916 ± 0.0015	0.348 ± 0.012	0.314 ± 0.015

Validation and test values. All AUCs are higher on the test set (0.8861–0.9035) than on the validation set (0.8584–0.8731). This is not a sign of leakage, since the split is by vehicle: the test set, with only 96 lane changes, happens to be easier. Absolute values are therefore not comparable across sets, and we compare architectures only within the same set.

Comparison with the baseline. The test set confirms that the baseline is dominated by smaller networks. 16-128-16-128, with 30% fewer parameters, improves the AUC by 0.0104 (about 6 standard errors) and the AP by 0.129, i.e. 39% in relative terms (about 24 standard errors). 16-32-16-16, 3.6 times smaller, improves the AUC by 0.0107 (about 4 standard errors) and the AP by 0.103 (about 13). 16-16-16, 4.8 times smaller, improves the AP by 0.089 (about 5 standard errors), but its AUC gain is not significant (0.0054, about 1 standard error): it is the least stable network, with one poor seed (AUC 0.8862, AP 0.349). The largest network does not improve on the baseline: its AUC is the same (-0.0002), and its AP gain (0.020, about 1.5 standard errors) is not significant. The single-layer network remains the weakest.

Plateau and knee. From 3,297 to 88,593 parameters, all architectures on the fronts have test AUCs between 0.9009 and 0.9035, indistinguishable given the noise. On the AP, 16-128-16-128 improves on 16-32-16-16 by 0.026 (about 3.5 standard errors), whereas 16-256-256-64, ten times larger, adds only 0.005, which is not significant. The rise of the AP up to about 8,500 parameters, suggested on the validation set, is thus confirmed on independent data.

ROC and PR curves. Fig. 6 shows where the gain lies. The ROC curves of the baseline and of two networks on the front almost coincide, consistently with their close AUCs. The PR curves, in contrast, separate clearly at low recall, i.e. among the windows that each network scores highest: there, the networks found by the search reach precisions of 0.8–0.9, against about 0.6 for the baseline. At higher recall the curves converge. As discussed in Sec. 3.1, with rare positives a small difference in FPR can correspond to a large difference in precision. With the threshold set to detect half of the positive windows, the baseline produces about 2.1 false-positive windows for every correct detection, 16-128-16-128 about 1.3 (37% fewer), 16-32-16-16 about 1.45, and the largest network 2.2.

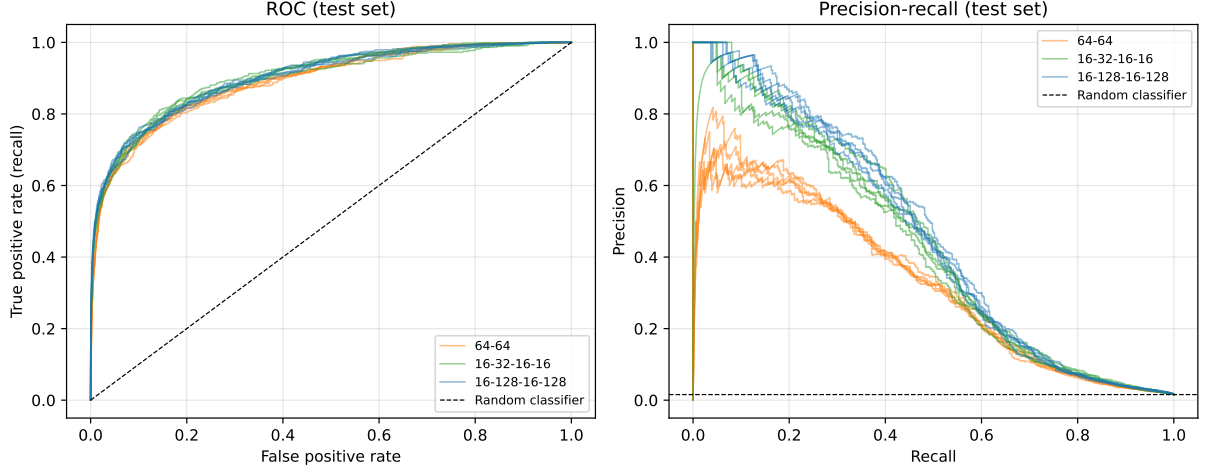


Figure 6: ROC (left) and precision–recall (right) curves on the test set for the baseline and two architectures on the front; one thin line per seed. Dashed: random classifier.

4.5 Control on straddling windows

The 468 straddling windows of the test set, which contain a lane crossing and were never used for training, serve as a control. For each network, we compute the AUC obtained by ranking these windows against the negative test windows. For all nine architectures this AUC lies between 0.9423 and 0.9840, above the ordinary test AUC (0.8861–0.9035). The median score of the straddling windows (0.96–0.98) is higher than that of the positive windows (0.87–0.94) and far above that of the negative ones (0.07–0.15). Although trained only on the weak cues that precede a manoeuvre, the networks recognise manoeuvres in progress even more easily. The differences between architectures on this control set are discussed in Sec. 5.

5 Discussion and limitations

Two results stand out: beyond a minimum depth, larger networks buy no measurable AUC, and the hand-designed baseline is dominated by networks several times smaller. Both hold on the test set, which was used only once, so they are not an artefact of selecting epochs and architectures on the validation set. This section discusses possible explanations and the limits of the study.

5.1 Why the quality saturates early

A plausible explanation of the plateau is that the quality is limited by the information contained in the input windows rather than by the capacity of the networks. A positive window ends between 0.1 and 3 s before the crossing, but the lateral displacement exceeds the noise level in only 8% of the lane changes at 3 s before the crossing and in 31% at 2 s (Table 1). Many positive windows therefore look like lane keeping, and no network can recover a cue that the input does not contain.

The same explanation accounts for the shape of the PR curves (Fig. 6). At low recall, the threshold selects the windows with the clearest lateral cues, and the precision is high. To reach higher recall, it must also accept positives with weak or no cues, and it then admits many lane-keeping windows as well: the precision drops and all networks converge. The advantage of the networks found by the search is thus concentrated where the cues are present.

Other factors may contribute. The 4,543 positive training windows come from only 486 lane changes, so the number of independent examples is small, which favours small models. All networks are trained without regularisation, and the larger ones overfit within a few epochs,

so the plateau may partly reflect the training protocol. Finally, the limit holds for the chosen features and for a flattened input. Our experiments cannot separate these effects.

5.2 The width of the first layer

Among the candidates with more than one layer, those with a first layer of 16 units are among the best, and the five weakest have a first layer of 64 or more units (Sec. 4.3). One interpretation is that a narrow first layer acts as a bottleneck: it compresses the 120 noisy inputs into a few features and limits the ability of the network to fit the noise, a form of implicit regularisation. An alternative is that the effect is one of size. The first layer alone has 121 h_1 parameters, about 31,000 for a width of 256, so a network with a wide first layer is necessarily large; and large networks reach their best validation AUC within a few epochs and then overfit (Sec. 4.3). Their weakness could then be due to overfitting rather than to the first layer itself. The data favour the first reading: networks with a narrow first layer remain among the best even with 50,000–90,000 parameters (16–64–256–128, 16–256–256–64), although they too reach their best epoch early. Both, however, remain hypotheses.

5.3 AUC as the selection criterion

The AUC was chosen before the experiments because it depends neither on a threshold nor on the fraction of positives. In hindsight, the AP is more closely aligned with the operational objective, reliable positive predictions for a rare event, and would have been an equally defensible choice. As Secs. 4.3 and 4.4 show, the conclusions hold on both metrics; only the position of the knee changes. The AP of each network is measured at the epoch selected by the AUC, which may not maximise it, but this affects all networks in the same way.

5.4 Recognition of manoeuvres in progress

On the 468 straddling windows of Sec. 4.5, which contain a lane crossing and were excluded from training, the ranking of the networks is partly inverted: the three networks with the lowest test AUC (16, the largest network and the baseline) rank these windows above the negative ones best (AUC 0.97–0.98), whereas several networks on the front reach only 0.94–0.95.

A possible reason is that a straddling window whose crossing occurs near its beginning shows a vehicle already settling into its new lane, which will usually not change lane again in the following 3s: by the definition of the task, a low score for such a window is not an error, and more selective networks may assign one. The straddling windows thus confirm that all networks recognise manoeuvres in progress, but are not suited to rank the networks.

5.5 Limitations

- *Cost measure.* We measure cost by the number of parameters and not by energy or time on real hardware. It is an acceptable proxy within the scope of this project.
- *Search space.* Only MLPs with a flattened input were considered, which discards the temporal structure of the window.
- *Search.* The empirical Pareto set is based on 300 of the 780 architectures, and only the four best of each cost band, according to a noisy single-seed estimate, were re-evaluated.
- *Training protocol.* All networks share the same hyperparameters and no regularisation, so that the comparison isolates the architecture. A baseline trained with dropout or weight decay might narrow the gap: the results show that the search finds smaller and better architectures under a fixed protocol, not that they beat a fully tuned model.

- *Data.* The data cover one freeway segment and 1,725 vehicles from the period in which congestion builds up; the test set contains only 96 lane changes. Left and right lane changes are pooled, and windows of merging vehicles are labelled as negatives.
- *Task design.* The window and horizon lengths, the class ratio of the training set and the features were fixed a priori and not varied.
- *Evaluation.* The metrics are computed on windows, not on lane changes, and each lane change produces several positive windows. The precision therefore measures the fraction of correct positive windows, not the fraction of correct alarms of a real system, which would raise one alarm per manoeuvre. It is adequate to compare the networks, which are all evaluated in the same way.

6 Conclusion and outlook

We applied a bi-objective neural architecture search, maximising the validation AUC and minimising the number of parameters, to the early recognition of lane changes on NGSIM data. Within a space of 780 MLPs, random search evaluated 300 architectures; the most promising ones were re-evaluated with five seeds, and the resulting Pareto front was checked once on the test set.

The search found networks that are both smaller and better than a hand-designed baseline. On the test set, 16-32-16-16, 3.6 times smaller than the baseline, is better on both AUC and AP, and 16-128-16-128, with 30% fewer parameters, improves the AP by 39% and produces about 37% fewer false-positive windows at a recall of 50%. Beyond about 8,500 parameters, nothing is gained: the largest network, with 19 times the parameters of the baseline, is no better than it. Within this family of models, most of the cost can be removed without loss of quality.

A methodological result emerged along the way. The variability between training runs with different seeds is of the same order as the differences between architectures, so a single run per architecture is enough to discard poor architectures but not to rank good ones, and selecting the best single-run result overestimates it. Re-evaluating the candidates with new seeds was necessary to obtain a reliable front.

Outlook. Several extensions follow from the limitations of Sec. 5:

- *Measured cost.* Replacing the number of parameters with energy and latency measured on the target hardware. The number of parameters does not necessarily reflect these quantities, as the training times of Sec. 3.2 show.
- *Richer search spaces.* Including one-dimensional convolutional networks, which exploit the temporal structure of the window, with the number of operations as cost measure.
- *Search strategy and objectives.* Using multi-objective search strategies [1], and adding the AP as a further objective or selection criterion.
- *Task and data.* Repeating the study with other window and horizon lengths, and verifying the explanation of the plateau (Sec. 5.1) by measuring which fraction of the positive windows is detected as a function of how long before the crossing they end.

Finally, a Pareto front can be seen as a collection of models with different costs and qualities, from which a system can choose at run time according to the available resources, for instance the energy budget. Such a collection is useful only if paying more buys more quality. In this task it does not: beyond a few thousand parameters the front is flat, and the collection reduces in practice to the smallest network and the one at the knee. Choosing models at run time becomes valuable on tasks where quality keeps growing with cost, and when cost is measured on real hardware.

Use of AI tools. An AI assistant (Claude, Anthropic) was used throughout this project. It drafted most of the code and proposed drafts of the text. All decisions on the study were taken by the author, who also ran all experiments, verified the results against the code and its outputs, and reviewed and revised the code and the text.

References

- [1] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search. In Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren, editors, *Automated Machine Learning: Methods, Systems, Challenges*, pages 63–77. Springer, 2019.
- [2] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023.
- [3] U.S. Department of Transportation, Federal Highway Administration. Next generation simulation (NGSIM) vehicle trajectories and supporting data. <https://data.transportation.gov/Automobiles/Next-Generation-Simulation-NGSIM-Vehicle-Trajectory/8ect-6jqj>, 2016. I-80 dataset.

A Lane-relative lateral position

Fig. 7 illustrates why we use the absolute lateral position rather than a position relative to the lane centre (Sec. 2). The left column shows a lane change, the right column a vehicle that approaches the lane boundary and returns without changing lane. Relative to the centre of the current lane, the position jumps by about one lane width at the crossing (middle row). Its absolute value removes the jump, but makes the two manoeuvres look alike (bottom row): in both cases it rises towards half a lane width and falls back, although only the first is a lane change.

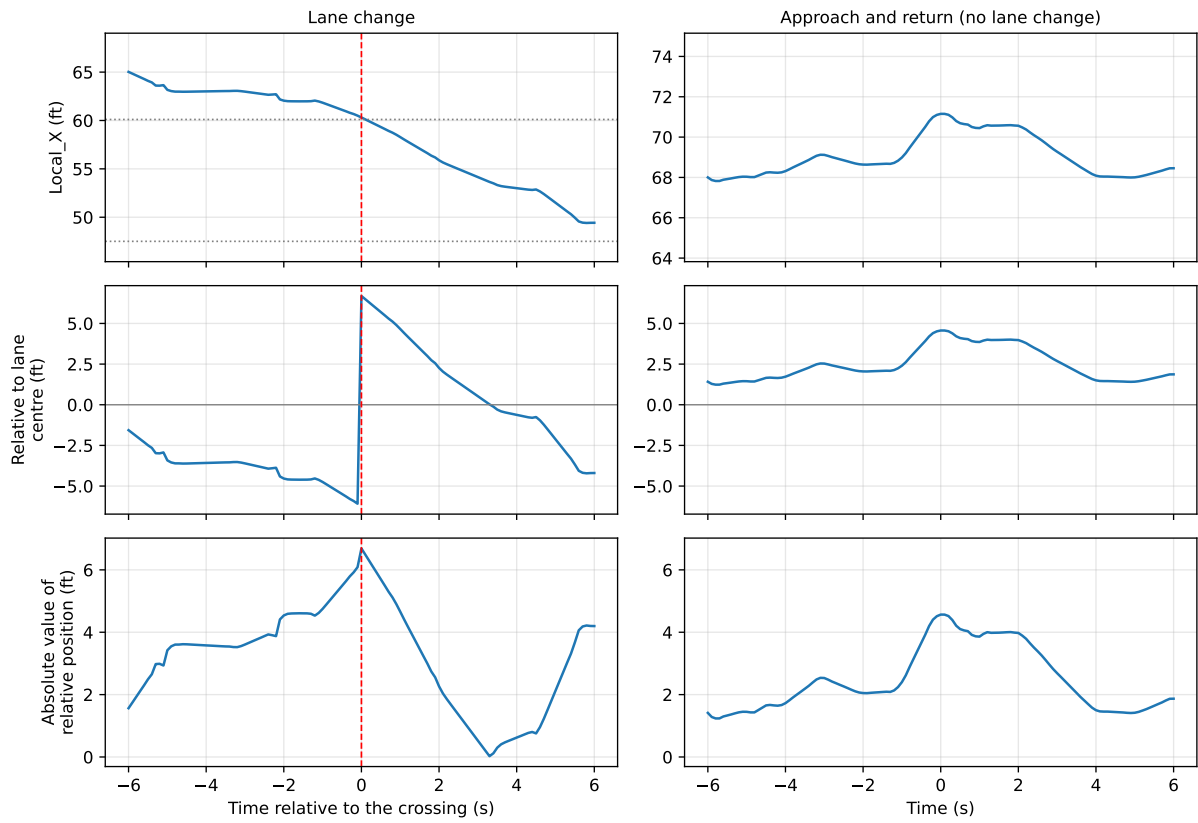


Figure 7: Three representations of the lateral position for a lane change (left; dashed red line: crossing) and for an approach to the lane boundary followed by a return (right). Top: absolute position $Local_X$, with the lane boundaries (dotted). Middle: position relative to the centre of the current lane. Bottom: absolute value of the relative position.

B Seed study

Table 10 reports the six architectures of the seed study, each trained with seeds 0–4. They were chosen to cover the range of sizes and results of the search: the smallest network, the baseline, the largest network, an intermediate one, the best network of the search (**16-16-64**) and a small network with a high single-seed AUC (**16-16-16**).

Table 10: Seed study: validation results over seeds 0–4. The AUC is reported as mean, standard deviation and range over the five seeds.

Architecture	Parameters	Mean AUC	Std. dev.	AUC range	Mean AP	Best epochs
16	1,953	0.8584	0.0014	0.8569–0.8602	0.309	19, 13, 46, 13, 26
16-16-16	2,497	0.8701	0.0032	0.8659–0.8745	0.377	15, 33, 15, 19, 10
16-16-64	3,361	0.8712	0.0053	0.8654–0.8795	0.392	17, 24, 18, 14, 9
32-32-32	6,017	0.8648	0.0044	0.8601–0.8705	0.357	12, 5, 6, 4, 4
64-64 (baseline)	11,969	0.8631	0.0010	0.8615–0.8642	0.342	6, 4, 7, 6, 5
256-256-256-256	228,609	0.8635	0.0040	0.8600–0.8702	0.368	4, 3, 3, 5, 3

C Re-evaluation of the candidates

Table 11 reports the 30 candidates of the re-evaluation phase, trained with seeds 1–5. The column “Drop” is the difference between the single-seed AUC of the search (seed 0) and the mean over seeds 1–5: a positive value means that the architecture performs worse with new seeds.

Table 11: Re-evaluation phase: validation AUC and AP, mean $\pm$ standard error over seeds 1–5; AUC of the search phase (seed 0); drop between the two; mean best epoch. Fronts: A = Pareto front on the AUC, P = Pareto front on the AP.

Architecture	Parameters	AUC	AP	AUC (seed 0)	Drop	Epoch	Fronts
16	1,953	0.8584 ± 0.0006	0.311 ± 0.005	0.8593	0.0009	24.4	A, P
16-16-16	2,497	0.8706 ± 0.0014	0.385 ± 0.011	0.8688	−0.0018	19.2	A, P
16-32-16-16	3,297	0.8714 ± 0.0020	0.395 ± 0.009	0.8694	−0.0020	20.4	A, P
16-16-64	3,361	0.8688 ± 0.0012	0.392 ± 0.002	0.8795	0.0107	16.4	
16-32-32	3,569	0.8665 ± 0.0016	0.371 ± 0.009	0.8714	0.0049	11.0	
16-32-32-64	5,713	0.8698 ± 0.0017	0.401 ± 0.003	0.8707	0.0009	16.4	P
32-32-32	6,017	0.8646 ± 0.0018	0.354 ± 0.010	0.8705	0.0059	6.2	
32-16-128	6,705	0.8697 ± 0.0012	0.389 ± 0.005	0.8716	0.0019	9.4	
16-32-32-128	7,889	0.8699 ± 0.0012	0.393 ± 0.013	0.8736	0.0037	11.0	
16-128-16-128	8,481	0.8718 ± 0.0017	0.413 ± 0.004	0.8754	0.0036	15.4	A, P
16-32-128-32	10,865	0.8698 ± 0.0014	0.399 ± 0.006	0.8752	0.0054	10.2	
64-64 (baseline)	11,969	0.8630 ± 0.0004	0.344 ± 0.002	0.8632	0.0001	5.4	
16-256-32	14,545	0.8669 ± 0.0019	0.397 ± 0.004	0.8757	0.0088	13.6	
32-16-128-64	14,897	0.8692 ± 0.0012	0.382 ± 0.006	0.8761	0.0068	6.0	
32-32-128-64	17,473	0.8657 ± 0.0027	0.386 ± 0.011	0.8721	0.0064	6.0	
64-32-256	18,529	0.8666 ± 0.0017	0.384 ± 0.006	0.8716	0.0050	5.6	
16-16-64-256	20,193	0.8689 ± 0.0027	0.395 ± 0.006	0.8719	0.0030	8.0	
32-32-64-256	23,937	0.8683 ± 0.0013	0.381 ± 0.004	0.8725	0.0042	4.4	
16-256-128-16	41,265	0.8676 ± 0.0017	0.383 ± 0.007	0.8711	0.0035	8.2	
32-32-256-128	46,401	0.8689 ± 0.0011	0.395 ± 0.006	0.8724	0.0035	5.8	
16-64-256-128	52,689	0.8731 ± 0.0021	0.400 ± 0.007	0.8747	0.0016	8.0	A
16-128-128-256	53,905	0.8706 ± 0.0008	0.401 ± 0.005	0.8718	0.0011	5.4	
64-128-128-256	65,857	0.8645 ± 0.0014	0.391 ± 0.012	0.8716	0.0071	4.6	
16-256-128-256	72,465	0.8702 ± 0.0021	0.408 ± 0.004	0.8710	0.0008	8.8	
32-256-256	78,369	0.8647 ± 0.0020	0.380 ± 0.008	0.8720	0.0073	4.2	
16-256-256-64	88,593	0.8718 ± 0.0005	0.416 ± 0.008	0.8702	−0.0016	8.4	P
256-256-64-256	130,113	0.8618 ± 0.0014	0.352 ± 0.014	0.8677	0.0060	3.6	
256-256-128-16	131,745	0.8611 ± 0.0011	0.329 ± 0.006	0.8671	0.0059	3.4	
32-256-256-256	144,161	0.8669 ± 0.0023	0.389 ± 0.005	0.8659	−0.0010	4.6	
256-256-256-256	228,609	0.8619 ± 0.0006	0.357 ± 0.008	0.8702	0.0083	3.2	

D Details of the protocol

Patience in the search phase. The search was first run with a fixed budget of 20 epochs per architecture. For an architecture whose best epoch was at most 10, the 20 epochs already include 10 epochs without improvement, so the patience rule of Sec. 3.2 gives the same result. The 41 architectures whose best epoch exceeded 10 were retrained with the patience rule. For 32 of them the result was unchanged; 5 narrow networks, which learn slowly, improved (for instance 16–16, from 0.8595 to 0.8638, with its best epoch at 31). For the remaining 4, the AUC decreased by 0.0007 to 0.0024: in the 20-epoch run their best epoch came late (epochs 17–20), after more than 10 epochs without improvement, so the patience rule stops the run earlier and keeps a previous, slightly lower peak. These differences are smaller than the variability between seeds. The search results reported in Sec. 4.1 include all the retrained values.

Test runs. The trained networks of the re-evaluation phase were not stored. The test runs were therefore obtained by retraining the nine architectures with the same seeds 1–5; in all 45 runs, the validation AUC matched that of the re-evaluation to four decimal places.